

C# 콘솔 게임 프로젝트

디벨로켓 메타버스 프로그래밍 14기 권혁찬

목차

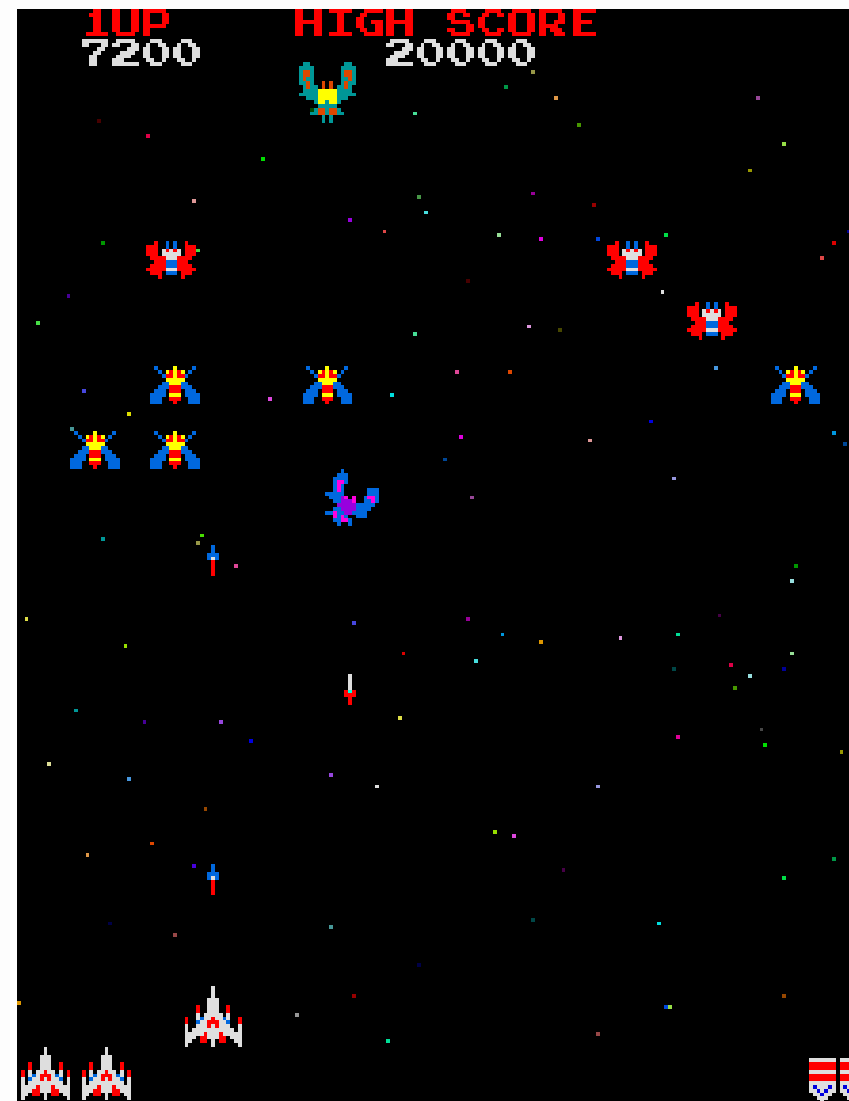
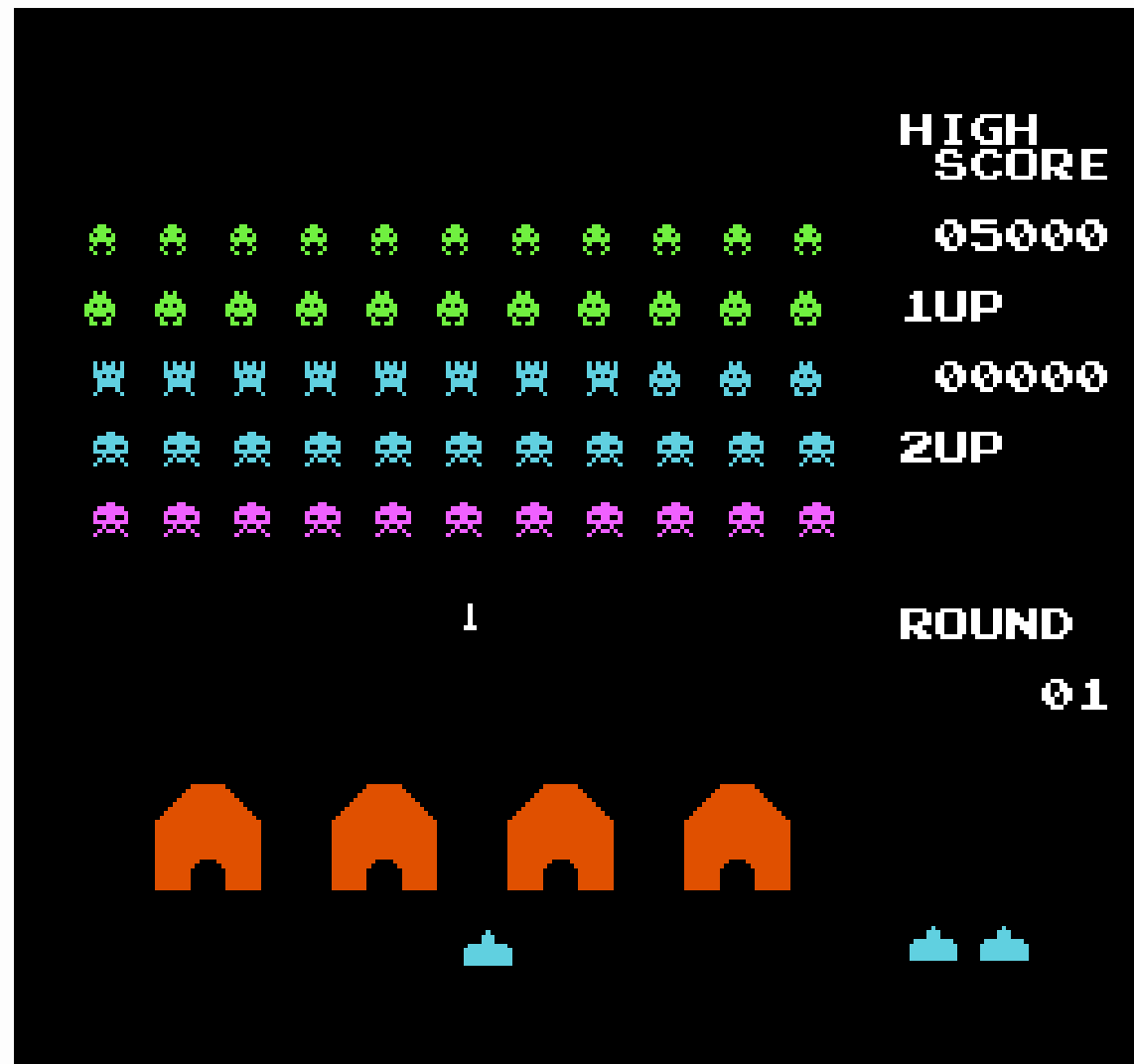
- 01 프로젝트 개요
- 02 기본 플레이 흐름
- 03 씬 구성
- 04 소스코드 구조
- 05 개발 기록

프로젝트 개요

-뭘 만들었는가?

윙스크롤 슈팅게임

고전적인 형태의 플라이트 슈팅게임을 콘솔 앱으로 구현하는걸 목표로 함

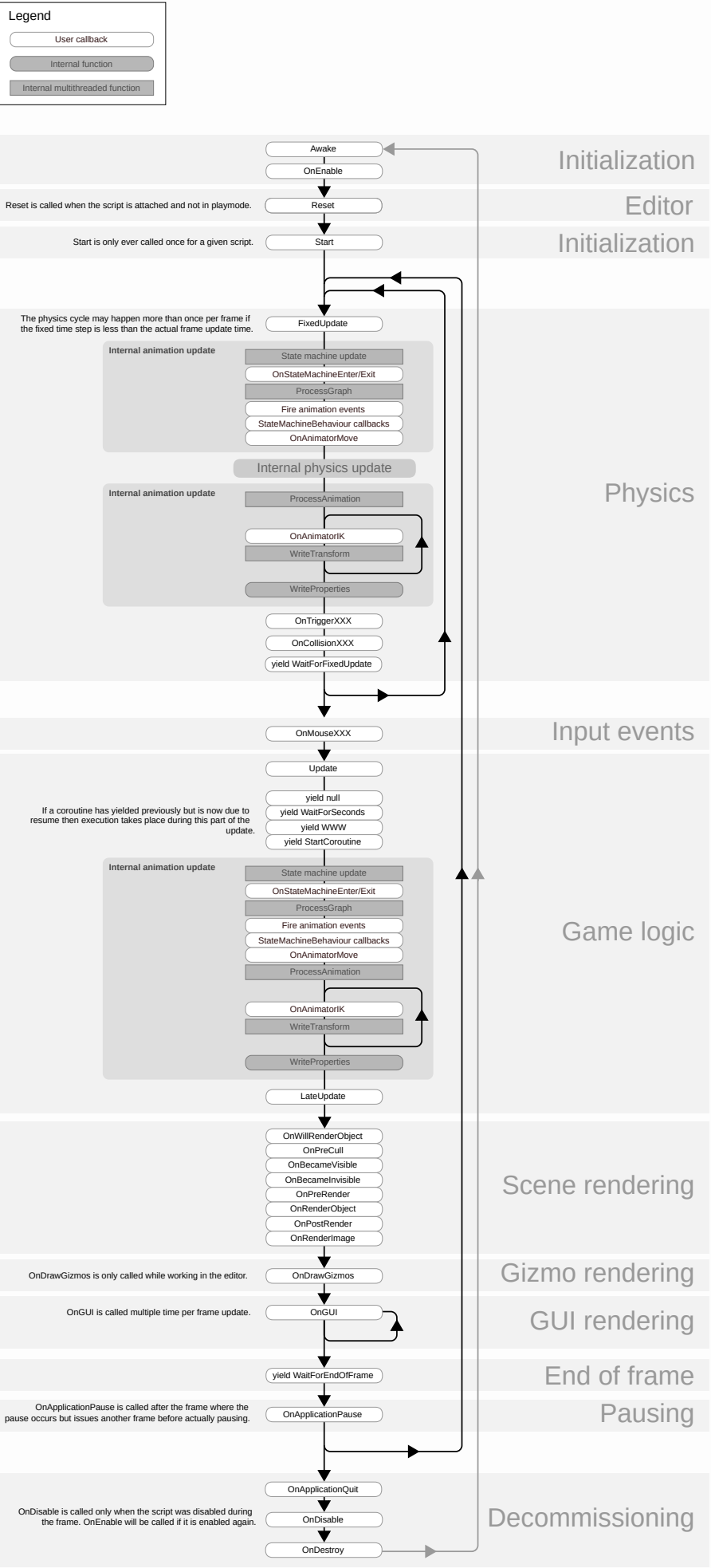


프로젝트 개요

-왜 만들었는가?

유니티 게임은 매 프레임마다 게임 내 개체들이 업데이트되어 상태가 변화하고, 화면이 갱신됨
이러한 프레임 시간당 발생하는 Update함수를 메인으로 한 유니티 게임의 생명주기 학습을 위해서 구현

+) 슈팅게임을 좋아함(중요)



기본 플레이 흐름

-메인 게임



-플레이어는 좌측에 배치. 적은 우측에서 출현

-상하좌우 방향키로 움직이며 Z키로 공격. X키로 폭탄 사용

-적을 처치하면 적 종류에 따라 점수를 획득

-적이나 적 탄환에 닿으면 게임오버

-화면에 적이 남아있지 않고 현재 스테이지에 더이상 소환될 적도 없다면 스테이지 클리어, 다음 스테이지로 넘어감

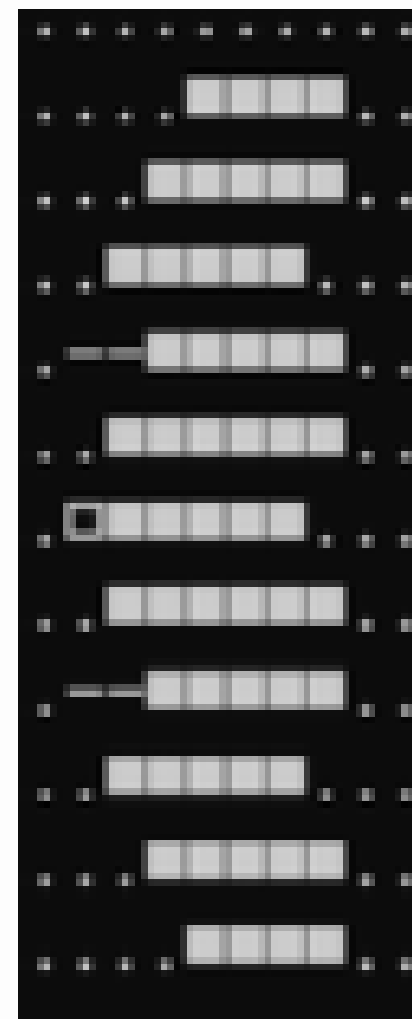
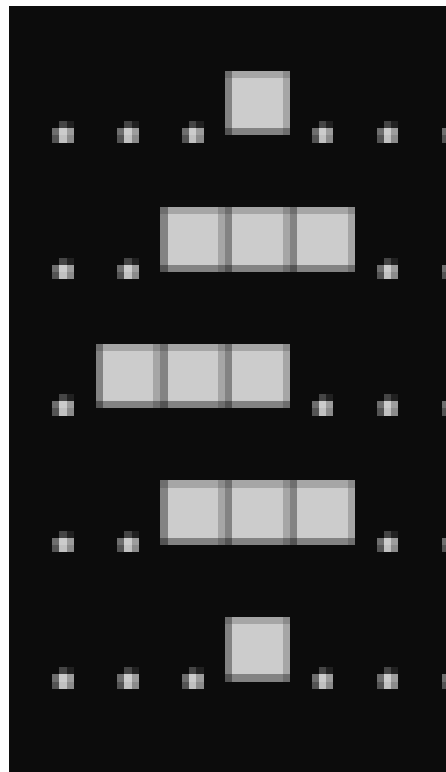
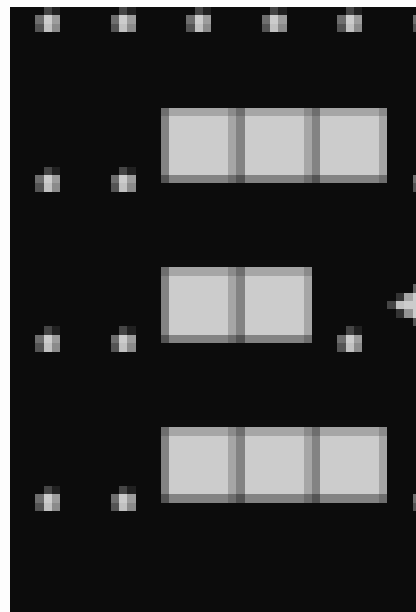
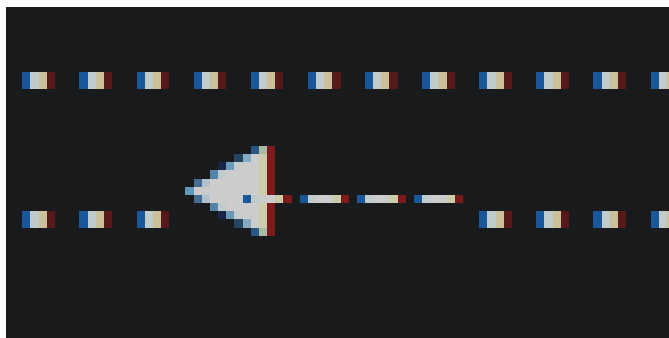
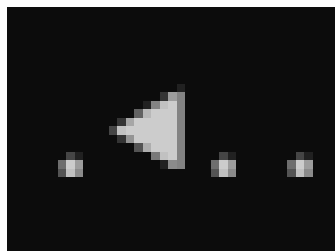
-총 5스테이지로 구성되어 있으며, 게임오버 되거나 5스테이지를 클리어하면 게임이 종료됨

기본 플레이 흐름

-적

총 5종류로 구성

각자 다른 체력과 피격 판정을 지님



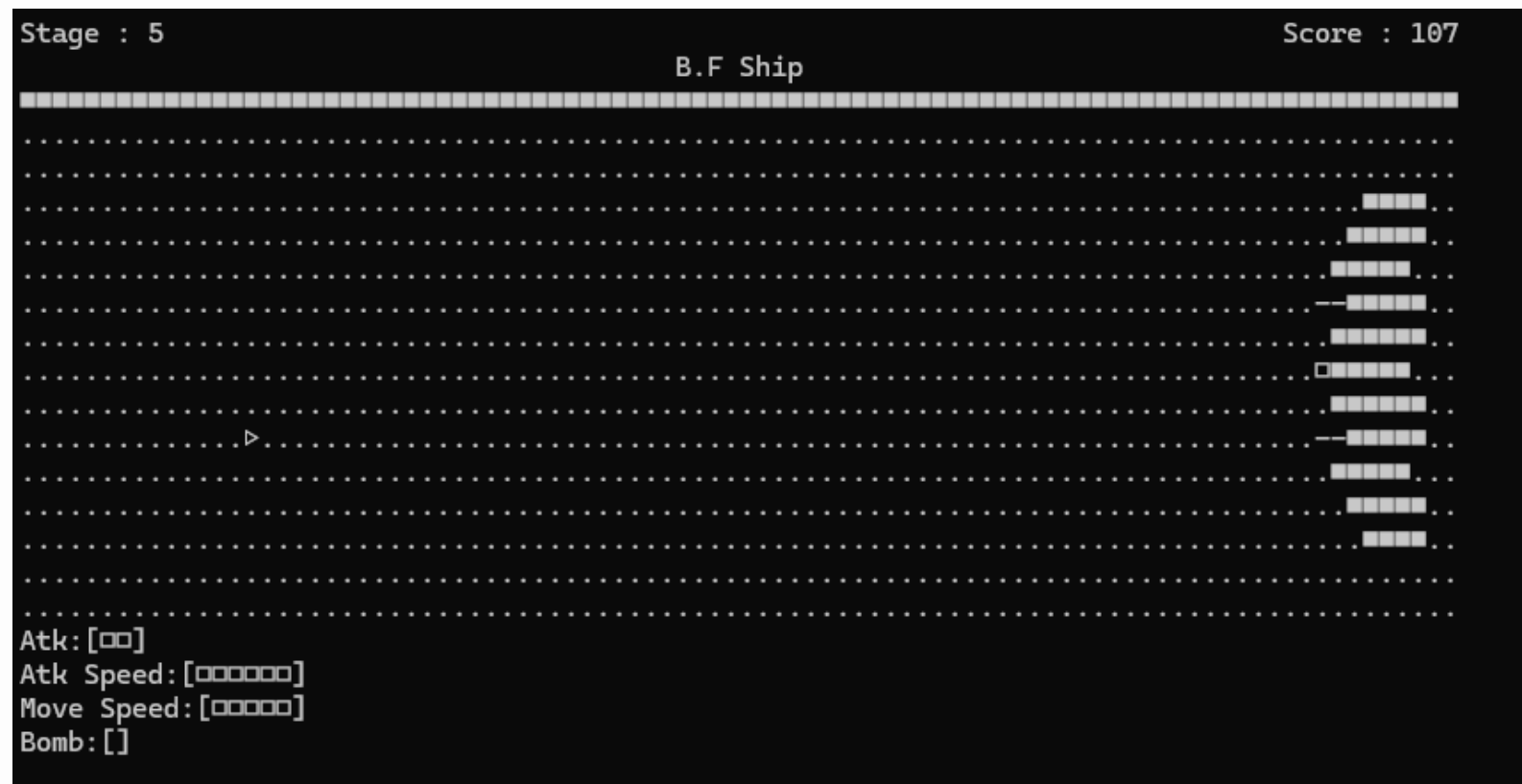
기본 플레이 흐름

-보스

5스테이지에 등장

화면 상단에 HP바가 표시됨

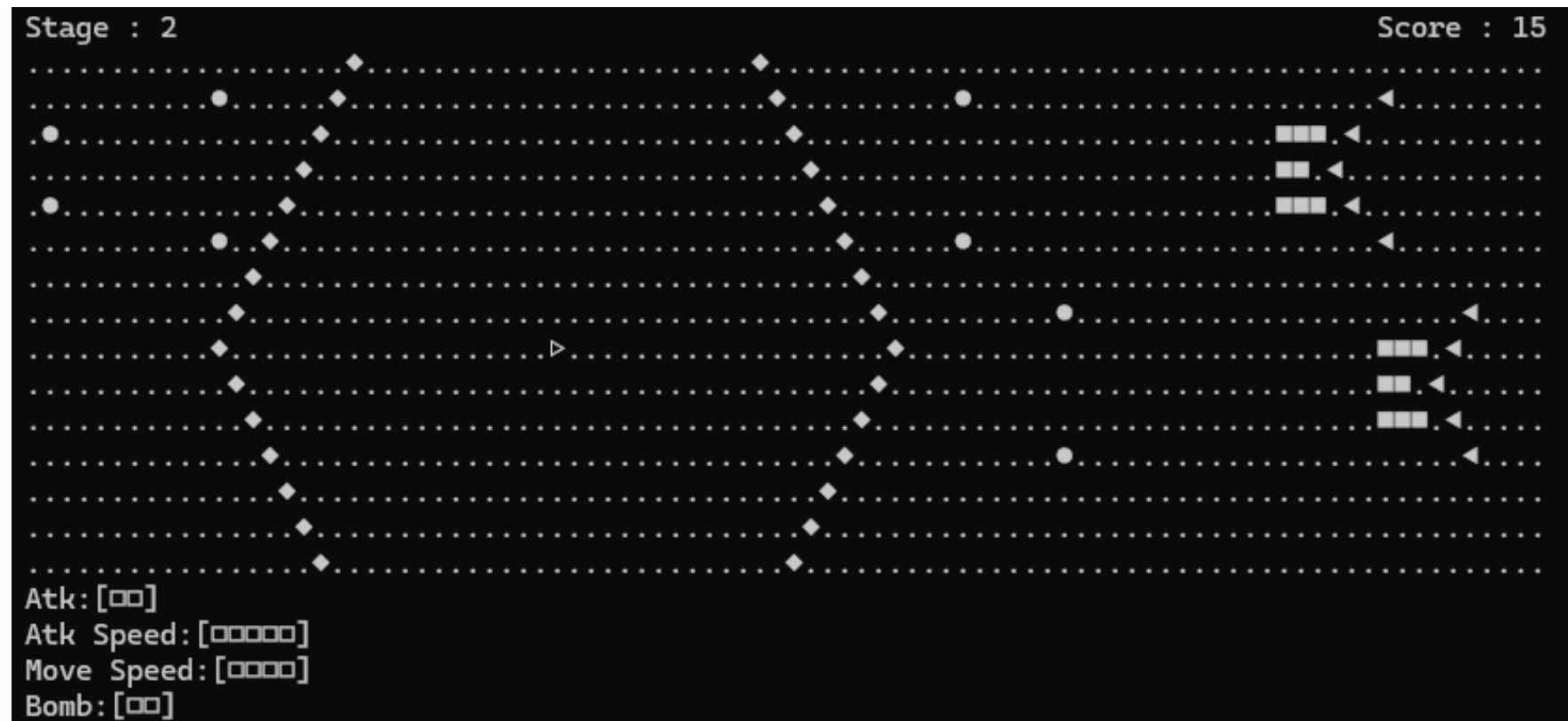
해당 스테이지는 보스 하나만 출현하므로 처치시 클리어



기본 플레이 흐름

-폭탄

사용시 퍼져나가는 연출과 함께 적 탄환을 지우고 적들에게 대량의 피해를 입힘
총 3개 지급

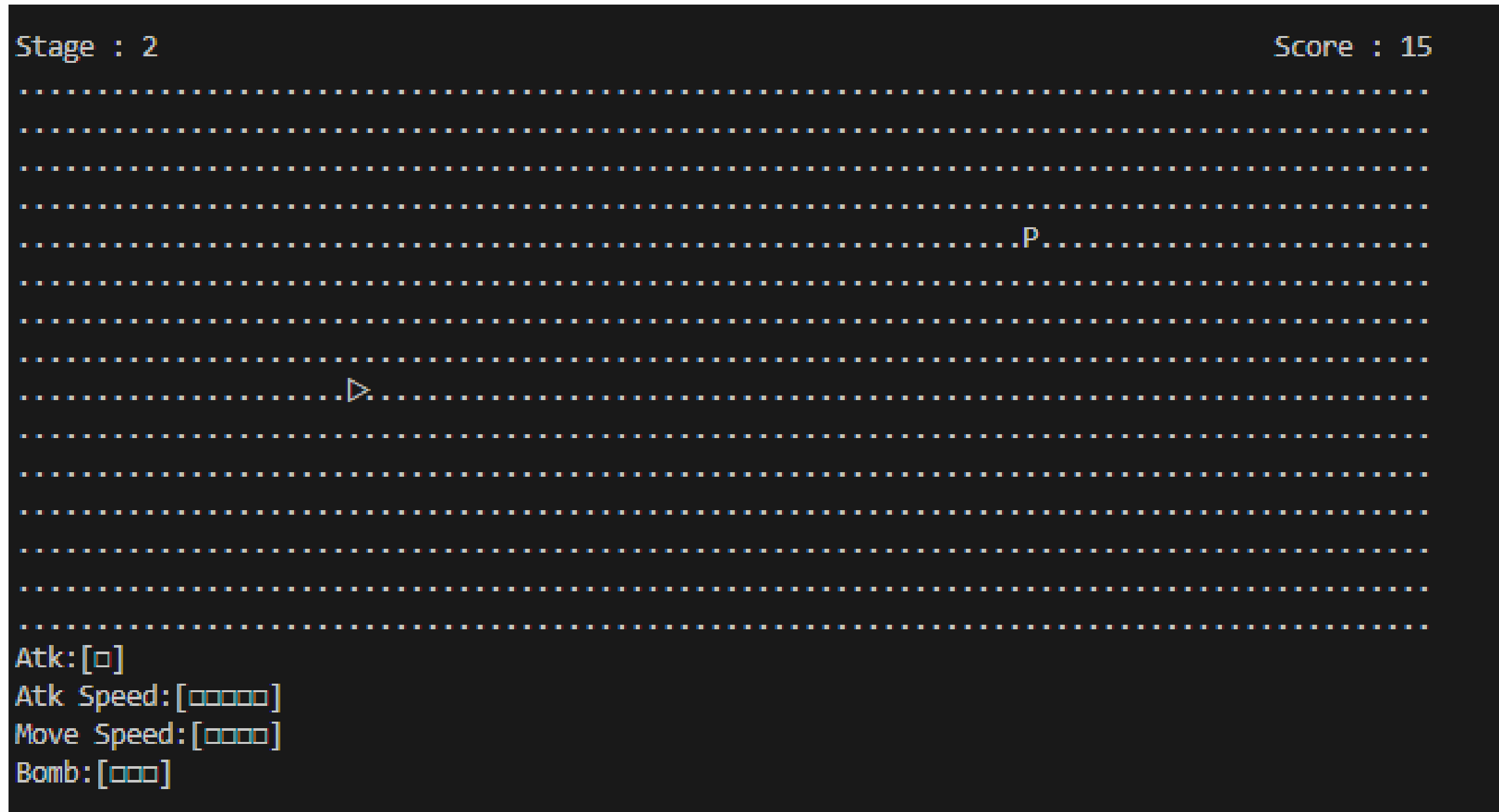


기본 플레이 흐름

-아이템

일부 적 처치시 맵 상에 드롭되어 움직임

플레이어가 획득 시 종류에 따라 공격력,공격속도,이동속도가 상승



기본 플레이 흐름

-게임오버

게임오버 시 연출 후 결과창으로 넘어감

획득 점수를 기록



Rank	Score	Name
1ST	00100	ABC
2ND	00035	ZZZ
3RD	00026	LOL
4TH	00003	AAA
5TH	00001	AAA
6TH	00000	RFJ
	00002	VAR

싹 구성

-전체 구조

총 6개의 싹으로 구성

-타이틀

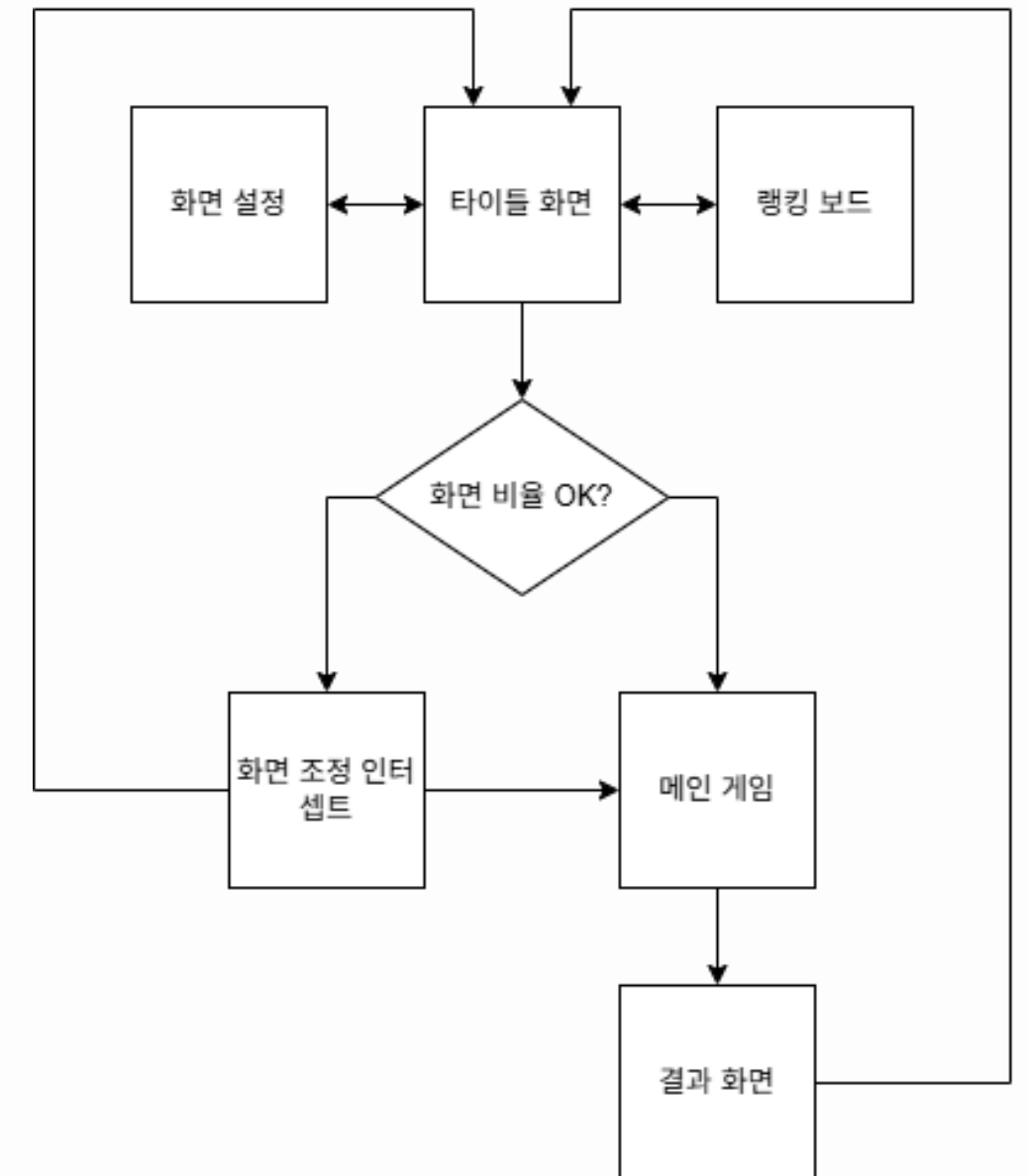
-화면 설정

-랭킹 보드

-게임 시작 전 화면 조정

-메인 게임

-결과



싹 구성

-타이틀

시작 화면. 게임을 시작하거나 화면 조정 싹, 혹은 랭킹 보드로 진입 가능

Shooting Game

- 1. Start Game
- 2. Screen Config
- 3. Ranking
- 0. Exit Game

썬 구성

-화면 설정

화면 크기가 권장 크기보다 작으면 UI상 문제가 생김

현재 화면 크기와 권장 화면 크기를 보여주면서 조정하는 썬

```
Recommand Size X = 100 Y = 30
```

```
Current Size X = 120 Y = 30
```

현재 사이즈가 권장 사이즈보다 크도록 화면 크기를 조정해주세요.

조정을 완료했다면 9번을 눌러 타이틀로 돌아가세요.

썸 구성

-게임 시작 전 화면 설정

권장 화면 크기보다 작은 상태로 게임을 시작하려할때 넘어가는 썸 화면 설정 썸과 거의 같으며, 무시하고 게임을 진행하는것도 가능

```
Recommand Size X = 100 Y = 30
```

```
Current Size X = 120 Y = 22
```

```
현재 스크린 사이즈가 권장 사이즈보다 작습니다.
```

```
1.게임시작하기
```

```
9.타이틀로 돌아가기
```

씬 구성

-메인 게임

실제 게임이 이루어지는 씬

좌상단에 현재 스테이지, 우상단에 점수, 중앙에 맵, 좌하단에 플레이어 기체 스탯이 표기됨



씬 구성

-결과,랭킹보드

결과창에서 게임 결과를 기록하고, 타이틀에서 랭킹보드로 진입하여 기록한 결과들을 확인 가능

Rank	Score	Name
1ST	00100	ABC
2ND	00035	ZZZ
3RD	00026	LOL
4TH	00003	AAA
5TH	00002	DKD
6TH	00001	AAA
7TH	00000	RFJ
	00002	

Rank	Score	Name
1ST	00100	ABC
2ND	00035	ZZZ
3RD	00026	LOL
4TH	00003	AAA
5TH	00002	DKD
6TH	00001	AAA
7TH	00000	RFJ

9.타이틀로 돌아가기

소스코드 구조

-업데이트 루프

게임의 모든 동작은 GameManager의 MainLoop 메소드 아래에서 돌아감

키 입력, 오브젝트 업데이트, 화면 렌더링까지 매 프레임 시간마다 프로그램 종료시까지 무한으로 실행됨

```
public class GameManager
{
    1 reference
    private async void MainLoop()
    {
        while(true)
        {
            //키 입력 액션 집합
            List<KeyAction> actions = new List<KeyAction>();
            GameState.Instance.CheckInput(actions);
            curScene.CheckInput(actions);
            InputProcessing(actions);

            //업데이트
            State.Update();
            curScene.Update();

            //화면 렌더링
            DrawScreen();

            //프레임 시간만큼 대기
            await Task.Delay(FrameTime);
        }
    }
}
```

소스코드 구조

-화면 그리기

현재 Scene에서 렌더링을 위한 데이터를 매 프레임마다 받아와서 화면을 덮어씀

글자를 매번 지우고 새로 쓰면 깜박임 현상이 일어나므로 Console.Clear는 최소화. 단순히 커서를 옮긴 후 일정 크기로 정의된 문자열을 그대로 출력해서 화면을 갈아치움

```
1 reference
private void DrawScreen()
{
    //화면 크기 이전 프레임과 다르면 지우개 켜빠나니까 한번 싹 밀고 다시 그리기
    if(lastScreenX != Console.WindowWidth || lastScreenY != Console.WindowHeight)
    {
        Console.Clear();
    }
    Console.SetCursorPosition(0, 0); //Write용 커서를 화면 좌상단으로 이동
    lastScreenX = Console.WindowWidth;
    lastScreenY = Console.WindowHeight;

    char[,] b = curScene.Render(); // 현재 씬에서 렌더링 데이터 받아오기
    StringBuilder sb = new StringBuilder();

    for(int y = 0 ; y < b.GetLength(0); y++)
    {
        int pointer = 0;
        for(int x = 0 ; x < b.GetLength(1) - pointer; x++)
        {
            sb.Append(b[y,x]);
            if(GetWidth(b[y,x]) == 2)
            {
                pointer += 1;
            }
        }
        sb.AppendLine();
    }
    Console.Write(sb.ToString()); //렌더링 데이터를 한줄한줄 출력
    Console.SetCursorPosition(0, 0);
}
```

소스코드 구조

-화면 그리기

렌더링 데이터는 현재 Scene 클래스에서 공백까지 포함하여 배열에 여러 규칙에 맞춰(화면 크기를 넘어서 안된다거나, 맵 오브젝트는 맵 좌표 내에서만 렌더링 되어야 한다거나) 쌓은 후 char[,] 타입으로 반환

2 references

```
public override char[,] Render()
{
    BufferClear();
    DrawString(0,0,"=====");
    DrawString(0,1,"      Shooting Game      ");
    DrawString(0,2,"=====");
    DrawString(0,3,"1.Start Game");
    DrawString(0,4,"2.Screen Config");
    DrawString(0,5,"3.Ranking");
    DrawString(0,6,"0.Exit Game");
    return buffer;
}
```

```
/// <summary>
/// 문자 한개 그리기
/// </summary>
/// <param name="x"></param>
/// <param name="y"></param>
/// <param name="txt"></param>
1 reference
protected void DrawChar(int x, int y, char txt)
{
    DrawString(x,y,txt.ToString());
}

/// <summary>
/// 문자열 한줄 그리기
/// </summary>
/// <param name="x"></param>
/// <param name="y"></param>
/// <param name="text"></param>
/// <param name="mX"></param>
/// <param name="mY"></param>
27 references
protected void DrawString(int x, int y, string text, int mX = 999, int mY = 999)
{
    if(y < 0) return;
    if(Math.Min(buffer.GetLength(0),mY) <= y) return;
    for (int i = 0; i < text.Length; i++)
    {
        if (x + i >= Math.Min(buffer.GetLength(1),mX) || x+i < 0)
            continue;
        buffer[y, x + i] = text[i];
    }
}

/// <summary>
/// 맵 오브젝트 그리기. 맵 좌표 내에서만 그려지도록 제한됨
/// </summary>
/// <param name="obj"></param>
5 references
protected void DrawObject(MapObject obj)
{
    int x = obj.Position.X+mapPos.X;
    int y = obj.Position.Y+mapPos.Y;
    DrawSString(x,y,obj.RenderShape(), GameState.MapSizeX+mapPos.X,GameState.MapSizeY+mapPos.Y);
}

4 references
protected void DrawSString(int x, int y, string[] ttext, int mX = 999, int mY = 999)
{
    for(int i = 0 ; i < ttext.Length; i++)
    {
        DrawString(x,y+i,ttext[i],mX,mY);
    }
}

8 references
```

소스코드 구조

-맵 오브젝트

플레이어, 적, 아이템 모두 MapObject 클래스를 상속함

맵 내에서 위치인 Position값 (Vector2 : X 좌표와 Y 좌표를 담은 구조체 타입), 오브젝트의 충돌 판정을 정의하는 크기값 GetSize, 렌더링되는 형태를 정의하는 RenderShape, GameState에서 매 프레임 호출하는 Update 메소드를 기본으로 지님

```
public abstract class MapObject
{
    /// <summary>
    /// 오브젝트의 위치. 좌상단 기준
    /// </summary>
    69 references
    public Vector2 Position{get;set;}
    /// <summary>
    /// 충돌 체크 시 해당 오브젝트의 크기 판정. RenderShape랑 별개로 운용
    /// </summary>
    /// <returns></returns>
    18 references
    public virtual Vector2 GetSize()
    {
        return new(1,1);
    }
    11 references
    public virtual void Move(Vector2 additive)
    {
        this.Position += additive;
    }
    4 references
    public virtual void Teleport(Vector2 moveTo)
    {
        this.Position = moveTo;
    }
    15 references
    public virtual void Update()
    {

    }
    /// <summary>
    /// 오브젝트 외형. 맵에 해당 외형대로 렌더링됨. 좌상단 기준
    /// </summary>
    /// <returns></returns>
    15 references
    public abstract string[] RenderShape();
}
```

소스코드 구조

-플레이어

키 입력 메소드인 CheckInput을 정의해 상하좌우 이동, 공격, 폭탄 사용에 대한 입력 결과를 정의

Update에서 적 탄환에 한정해 충돌 판정 처리. 충돌 시 게임오버 연출로 연결

맵 위에서 존재하기 위한 여러 기본적인 동작들은 MapObject에 정의되어 있으므로 Player의 내용은 이정도가 전부

```
public class Player : MapObject, IInputable
{
    public override string[] RenderShape()
    {
        return [">"];
    }
}
```

```
2 references
public void CheckInput(List<KeyAction> actions)
{
    actions.AddRange(
    [
        new (ConsoleKey.LeftArrow, () => MoveAction(ConsoleKey.LeftArrow)),
        new (ConsoleKey.RightArrow, () => MoveAction(ConsoleKey.RightArrow)),
        new (ConsoleKey.UpArrow, () => MoveAction(ConsoleKey.UpArrow)),
        new (ConsoleKey.DownArrow, () => MoveAction(ConsoleKey.DownArrow)),
        new (ConsoleKey.Z, () => Shoot()),
        new (ConsoleKey.X, UseBomb)
    ]);
}
```

```
public class Player : MapObject, IInputable
{
    public override void Update()
    {
        base.Update();
        coolTimer.Update();
        CheckHit();
    }
    1 reference
    public void CheckHit()
    {
        var bList = GameState.Instance.GetBulletList().FindAll(x => x.faction == Faction.Enemy);
        foreach(var b in bList)
        {
            if(GameUtil.CheckHit(this,b))
            {
                Hit();
                return;
            }
        }
        var eList = GameState.Instance.GetEnemyList();
        foreach(var e in eList)
        {
            if(GameUtil.CheckHit(this,e))
            {
                Hit();
                return;
            }
        }
    }
}
```


소스코드 구조

-적

최대 HP, 처치 시 점수, AI(행동 패턴), 드랍아이템으로 구성

플레이어 탄환과 충돌 체크하여 충돌 시 플레이어 스탯에 기반해 피해를 입음

HP와 처치 시 점수는 파생 클래스 내에서 동일. AI와 드랍아이템은 개체 생성 시 초기화

같은 외형을 지니고 같은 클래스로부터 파생되었지만 다른 행동패턴이나 드랍 아이템을 지니게 하기 용이함

```
public abstract class Enemy : MapObject
{
    1 reference
    public virtual void Init(EnemyAI ai, Item d)
    {
        HP = MaxHP;
        this.enemyAI = ai;
        dropItem = d;
        ai.Init(this);
    }
    3 references
    public virtual bool IsBoss => false;
    2 references
    public virtual string Name => "Enemy";
    6 references
    public virtual int Score => 1;
    6 references
    public virtual int HP{get;set;}
    8 references
    public abstract int MaxHP{get;}
    7 references
    public override abstract string[] RenderShape();
    8 references
    public override void Update()
    {
        base.Update();
        BulletCheck();
        if(enemyAI != null)
        {
            enemyAI.Update();
        }
    }
}
```

```
/// <summary>
/// 잡몹
/// </summary>
25 references
public class Jaco : Enemy
{
    2 references
    public override int Score => 1;
    4 references
    public override int MaxHP => 1;
    3 references
    public override string[] RenderShape()
    {
        return ["◀"];
    }
}
```

```
/// <summary>
/// 몸뚱이
/// </summary>
2 references
public class Shielder : Enemy
{
    2 references
    public override int Score => 5;
    4 references
    public override int MaxHP => 30;

    13 references
    public override Vector2 GetSize()
    {
        return new Vector2(3,3);
    }
    3 references
    public override string[] RenderShape()
    {
        return[
            "███",
            "███",
            "███"
        ];
    }
}
```

소스코드 구조

-적 AI

적의 행동패턴을 정의하는 클래스

적 개체마다 다르게 설정 가능

```
/// <summary>
/// 적 AI 클래스
/// </summary>
13 references
public abstract class EnemyAI
{
    1 reference
    public virtual void Init(Enemy o)
    {
        owner = o;
    }
    9 references
    public virtual void Update()
    {
        OutCheck();
    }
    /// <summary>
    /// 기본 맵 외각 탈출 체크 코드. 오브젝트 사이즈 보장 있음. 항상 좌측을 향해 이동하므로 우측 바깥으로는 체크 안함
    /// </summary>
    1 reference
    protected void OutCheck() ...
    42 references
    public Enemy owner;
}
```

```
public class MoveAndAttack : EnemyAI
{
    /// <summary>
    /// 좌측으로 계속 이동하면서 전방으로 쏘는 AI
    /// </summary>
    /// <param name="moveSpeed"></param>
    /// <param name="attackSpeed"></param>
    25 references
    public MoveAndAttack(float moveSpeed, float attackSpeed)
    {
        coolTimer = new CoolTimer();
        //moveSpeed *= GameUtil.RandomRange(0.9f,1.1f);
        //attackSpeed *= GameUtil.RandomRange(0.9f,1.1f);
        coolTimer.SetCool(Move,(int)(1000/moveSpeed),0,false,MoveTime);
        coolTimer.SetCool(Attack,(int)(1000/attackSpeed),0,false,AttackTime);
    }
    6 references
    public override void Update()
    {
        base.Update();
        coolTimer.Update();
    }
    1 reference
    public void AttackTime()
    {
        ...
    }
}
```

```
new(new(100,0),new Jaco(),new MoveAndAttack(10,1)),
```


소스코드 구조

-아이템

일부 적을 처치 시 맵상에 드랍됨

일정 간격으로 대각선으로 이동. 맵 가장자리에 부딪히면 경로 반전

플레이어와 충돌하면 획득 메소드를 실행 후 삭제

위 동작이 모두 부모 클래스인 Item에 정의되어 있기 때문에 파생 클래스는 외형과 획득 시 효과만 정의하면 됨

```
public abstract class Item : MapObject
{
    0 references
    public Item()
    {
        coolTimer = new CoolTimer();
        coolTimer.SetCool(Move,100,100,false,MoveTime);
        coolTimer.SetCool(Remove,10000,0,true>DeleteItem);
    }
    1 reference
    public void Init(Direction dirLR, Direction dirUD)
    {
        directionLR = dirLR;
        directionUD = dirUD;
    }
    6 references
    public abstract override string[] RenderShape();
    //그래픽상 크기는 1x1이지만 아이템 수집 범위는 넓게 구현
    13 references
    public override Vector2 GetSize()
    {
        return new Vector2(5,5);
    }
    8 references
    public override void Update()
    {
        base.Update();
        coolTimer.Update();
        PlayerCheck();
    }
}
```

```
/// <summary>
/// 공격력 업 아이템
/// </summary>
1 reference
public class PowerUp : Item
{
    5 references
    public override void Earn()
    {
        base.Earn();
        GameState.Instance.PStat.AddAtk(1);
    }
    3 references
    public override string[] RenderShape()
    {
        return ["P"];
    }
}
```

```
/// <summary>
/// 이동속도 업 아이템
/// </summary>
1 reference
public class SpeedUp : Item
{
    5 references
    public override void Earn()
    {
        base.Earn();
        GameState.Instance.PStat.AddSpeed(5);
    }
    3 references
    public override string[] RenderShape()
    {
        return ["S"];
    }
}
```


소스코드 구조

-GameState

메인 게임 데이터(플레이어 정보, 맵 오브젝트 전부, 점수, 스테이지 정보 등)를 모두 지닌 클래스
GameManager와 함께 싱글톤 패턴으로 디자인됨(게임 내 유일한 객체, 코드 어디에서든 쉽게 접근 가능)

```
public class GameState : IInputable
{
    69 references
    public static GameState Instance {get; private set;}
    1 reference
    public GameState()
    {
        Instance = this;
        Init();
    }
    2 references
    public void Init()
    {
        Score = 0;
        PStat = new PlayerStat();
        CurBoss = null;
        player = new Player();
        player.Position = new Vector2(0, MapSizeY/2);
        bulletPool = new List<Bullet>();
        enemyPool = new List<Enemy>();
        itemPool = new List<Item>();
        otherPool = new List<MapObject>();
        Wave = new WaveManager();
        Wave.DataInit();
    }
}
```

```
public class GameState : IInputable
{
    public void AddEnemy(Enemy enemy)
    {
        enemyPool.Add(enemy);
        if(enemy.IsBoss) RegisterBoss(enemy);
    }
    1 reference
    public void DeleteEnemy(Enemy enemy)
    {
        enemyPool.Remove(enemy);
        if(enemy.IsBoss) UnRegisterBoss();
    }
    1 reference
    public void AddItem(Item item)
    {
        itemPool.Add(item);
    }
    1 reference
    public void DeleteItem(Item item)
    {
        itemPool.Remove(item);
    }
    5 references
    public void AddObj(MapObject item)
    {
        otherPool.Add(item);
    }
    1 reference
    public void DeleteObj(MapObject item)
    {
        otherPool.Remove(item);
    }
}
```

개발 기록

-키 입력 문제

초당 수십번씩 화면이 새로 그려지기 때문에 일반적인 Console.ReadLine은 사용 불가능

그래서 초기엔 Console.ReadKey 로 입력을 확인

하지만 반복 입력까지 딜레이가 발생하는 문제(꼭 누르고 있을때 A---AAAAA 되는거)가 있어서 User32.dll 의 GetAsyncKeyState 함수를 가져와서 사용

```
public struct KeyAction
{
    9 references
    public KeyAction(ConsoleKey k, Action a)
    {
        key = [k];
        action = a;
    }
    8 references
    public KeyAction(ConsoleKey[] k, Action a)
    {
        key = k;
        action = a;
    }
    3 references
    public ConsoleKey[] key;
    3 references
    public Action action;
}
```

```
2 references
public void CheckInput(List<KeyAction> actions)
{
    actions.AddRange(
    [
        new (ConsoleKey.LeftArrow, () => MoveAction(ConsoleKey.LeftArrow)),
        new (ConsoleKey.RightArrow, () => MoveAction(ConsoleKey.RightArrow)),
        new (ConsoleKey.UpArrow, () => MoveAction(ConsoleKey.UpArrow)),
        new (ConsoleKey.DownArrow, () => MoveAction(ConsoleKey.DownArrow)),
        new (ConsoleKey.Z, () => Shoot()),
        new (ConsoleKey.X, UseBomb)
    ]);
}
```

```
//키 인풋을 위한 라이브러리.
[DllImport("user32.dll")]
1 reference
static extern short GetAsyncKeyState(int vKey);
```

개발 기록

-한글 문자 렌더링 문제

C#의 char는 한글과 영어 모두 16비트(2바이트)로 동일하지만, 콘솔창에 출력할때 영문자는 1칸을, 한글은 2칸을 차지함
문자를 한줄씩 공백을 포함하여(그래야 이전 프레임 렌더링 결과를 완전히 덮어쓰므로) 출력하는 로직상 한글 문자가 추가되면 렌더링 데이터인 char[,] 배열의 길이보다 실제 콘솔에 찍히는 문자열이 더 길어지게 되고, 줄바꿈이 일어남
이로인해 예상치 못한 출력결과가 나오고 최악의 경우 렌더링 데이터가 화면 바깥으로 넘어가 스크롤이 일어나면서 출력 결과가 망가지게 됨

해당 문제를 해결하기 위해 한글 문자를 기입할땐 버퍼를 2칸 소모하게끔 조정함

```
=====
      Shooting Game
=====
1.Start Game
2.Screen Config
3.Ranking
0.Exit Game
```

```
Recommand Size X = 100 Y = 30
Current Size X = 158 Y = 16
현재 스크린 사이즈가 권장 사이즈보다 작습니다.
```

```
1.게임시작하기
   ing
9.타이틀로 돌아가기
```

```
for(int y = 0 ; y < b.GetLength(0); y++)
{
    int pointer = 0;
    for(int x = 0 ; x < b.GetLength(1) - pointer; x++)
    {
        sb.Append(b[y,x]);
        if(GetWidth(b[y,x]) == 2)
        {
            pointer += 1;
        }
    }
    sb.AppendLine();
}
```

```
//한글이면 버퍼 2칸 차지해야함. 헛지피티 작품
1 reference
public static int GetWidth(char c)
{
    // 한글 음절
    if (c >= 0xAC00 && c <= 0xD7A3)
        return 2;

    // 한글 자모
    if (c >= 0x1100 && c <= 0x11FF)
        return 2;

    return 1;
}
```

개발 기록

-CoolTimer

N초마다 발동. M초후 동작 등의 기능을 구현하기 위해 Update마다 FrameTime만큼 변수값들을 증감하다가 고안함

이런 ‘쿨’이 찰때마다 작동하는 동작들을 하나의 클래스로 묶음

Update에서 이런 변수들을 모두 제어할 필요없이 CoolTimer에게 Update만 호출하면 알아서 관리함

```
public Player()
{
    coolTimer = new CoolTimer();
    coolTimer.SetCool(shootCool, 1000/GameState.Instance.PStat.ShotSpeed, 1000, false, null);
    coolTimer.SetCool(moveCool, 1000/GameState.Instance.PStat.Speed, 1000, false, null);
    coolTimer.SetCool(bombCool, 2000, 0, false, null);
}
```

```
public class Player : MapObject, IInputable
{
    public override void Update()
    {
        base.Update();
        coolTimer.Update();
        CheckHit();
    }
}
```

```
public void Shoot()
{
    if(!coolTimer.IsCoolComp(shootCool)) return;
    GameState.Instance.ShootBullet(this.Position+new Vector2(1,0), Direction.Right, 100, Faction.Player);
    coolTimer.RefreshCool(shootCool, 1000/GameState.Instance.PStat.ShotSpeed);
}
```